

Digital Campus Platform

What we built this iteration — and how to run & deploy it

Prepared for the FSB Nexus team. Covers the chat fix, landing page, real dashboard, onboarding, security hardening, database migrations and CI, plus local setup, Google Cloud setup for Google Classroom, and step-by-step deployment on Google Cloud.

How to read this document. Each feature is explained twice: a short **In plain words** paragraph for anyone, and an **Under the hood** paragraph for developers. Sections 3–5 are practical, copy-paste setup and deployment guides.

Contents

- 1 Executive summary
- 2 What changed, feature by feature
- 3 Running the platform locally
- 4 Google Cloud setup for the Google Classroom integration
- 5 Deploying the platform on Google Cloud — step by step
- 6 Post-deployment checklist

1. Executive summary

In plain words. The assistant now actually answers in the app and does so quickly. New visitors get a polished landing page that explains the product, and once signed in they go through a short setup (“onboarding”) and land on a dashboard that shows *their own real* data instead of placeholder numbers. We closed several safety gaps (pages can no longer be reached without logging in, repeated failed logins are throttled) and put the project on a professional footing with database migrations and automated tests that run on every change.

Under the hood. Six coherent changes landed on `develop`: (0) fixed the SSE streaming parser; (1) a client-rendered animated landing page; (2) dashboard/header/sidebar now fetch real profile, courses and chat data; (3) an onboarding endpoint that is the only way to become an *enrolled* student, which unblocks the Academic agent; (4) a Next.js middleware route guard, a readable auth cookie, live Redis rate limiting and structured logging; (5) Alembic migrations and a GitHub Actions CI pipeline.

Area	Before	After
Chat	Spinner never resolved	Streams tokens live, with citations
Landing page	Redirected straight to login	Professional animated public page
Dashboard	Hardcoded mock (fake GPA, “Jordan Doe”)	Real profile, courses, conversations
Account setup	No onboarding; could not become “enrolled”	Guided onboarding; enrolled unlocks Academic agent
Access control	Protected pages rendered then bounced	Server-side redirect before any HTML
Brute force	Rate limiter present but inert	Enforced (5 attempts / minute / IP)
Schema & QA	No migrations, no CI	Alembic migrations + CI on every push/PR

2. What changed, feature by feature

2.0 The chat now renders its answer (critical fix)

In plain words. The chat used to sit on the loading dots forever even though the server was answering correctly. The reply now appears word-by-word, and its sources show underneath.

Under the hood. The server streams Server-Sent Events terminated with `\r\n\r\n` (sse-starlette's default). The browser parser split the buffer on `\n\n`, which never matched, so no event boundary was found. The fix splits on `/\r\n\r\n|\r\r|\n\n/`. No backend change was needed. File:

`frontend/src/lib/services/chat.ts`.

2.1 Public landing page

In plain words. Visitors who aren't signed in now see a real home page: a headline, an animated preview of the assistant answering with sources, a feature list, a “how it works” section and clear “Se connecter / Commencer” buttons.

Under the hood. `frontend/src/app/page.tsx` replaced a bare `redirect('/login')` with a client component using framer-motion and the existing design tokens. It is token-aware, fully responsive, works in light and dark mode, and respects `prefers-reduced-motion`. The mock streaming card lives in

`frontend/src/components/landing/ChatPreview.tsx`.

2.2 Real dashboard and identity

In plain words. The dashboard previously showed invented numbers (a fake GPA of 3.87, a fake name). It now shows the signed-in student's real name, status, their actual courses with document counts, their recent conversations, and an honest “profile completeness” figure. Empty states guide new users. The header gained a logout button.

Under the hood. `dashboard/page.tsx` is now a client component fetching `profileService.getProfile()`, `coursesService.listMine()` and `chatService.getSessions()` in parallel. The header (`((protected)/layout.tsx)` and `Sidebar.tsx` render the real profile, compute completeness from filled fields, and drop the hardcoded “Jordan Doe / Admin Blueprint / 84%” content and dead nav links.

2.3 Onboarding and the “enrolled” fix

In plain words. New users are guided once through a short form (status -> academic year -> baccalaureate -> interests). Choosing “enrolled student” matters: it lets the assistant use the specialised *Academic* agent for course/curriculum questions.

Under the hood. A new `POST /profile/onboarding` is the *only* endpoint that can set `student_status = "enrolled"` (admin stays rejected) and flips `onboarding_completed`. Previously no endpoint could set “enrolled”, so the `enrolled -> academic` routing branch was unreachable dead code. The protected layout gates unfinished users to `/onboarding`. We verified the newly-reachable Academic agent answers general, admission and curriculum questions — **0 refusals across 9 runs** — so routing needed no change.

2.4 Security hardening (pilot-solid)

In plain words. You can no longer briefly see a protected page before being kicked to login — the redirect now happens on the server. Hammering the login form with wrong passwords is now throttled. And the server keeps readable, timestamped logs so problems can be diagnosed.

Under the hood.

- **Route guard:** `frontend/src/middleware.ts` redirects unauthenticated requests for the protected paths to `/login` before any HTML ships. Middleware can't read `localStorage`, so `setToken` now also mirrors the JWT into a readable `token` cookie; logout and the axios 401 interceptor clear both to avoid bounce loops. It is a UX guard, not the security boundary — the real protection stays the API's 401 and the backend route guard.
- **Rate limiting:** a `docker-compose.yml` now provides Redis so the existing `/auth/login` limiter actually enforces (verified: 6th rapid bad login -> HTTP 429).
- **Logging:** `backend/app/main.py` configures root logging so application loggers surface with timestamps and levels.

2.5 Database migrations and CI

In plain words. The database structure is now version-controlled, so every environment can be recreated identically, and a robot runs the whole test suite automatically on every change to catch regressions before they merge.

Under the hood. Alembic is initialised (`backend/alembic/`) with `env.py` wired to `settings.DATABASE_URL` and `Base.metadata`, including correct rendering of the pgvector `Vector` type and a `CREATE EXTENSION vector` step. The initial migration was verified reversible (up->down->up) with no drift versus the models. GitHub Actions runs the AI suite (43 tests, mocked) and the backend suite (26 tests) against a pgvector service after applying migrations. The backend job runs without Redis on purpose: a live Redis would trip the shared-IP login limiter mid-suite; CI exercises the fail-open path instead.

3. Running the platform locally

You need Docker, Python 3.12, Node 20+, and a Mistral API key.

Step 1 — Infrastructure (Postgres + Redis)

```
docker compose up -d          # Postgres with pgvector on :5433, Redis on :6379
```

Step 2 — Python environment (backend + AI in one venv)

```
python -m venv .venv && . .venv/Scripts/activate # macOS/Linux: source .venv/bin/activate
pip install -r backend/requirements.txt -r ai/requirements.txt
```

The AI dependencies include torch and sentence-transformers; the multilingual embedding model (~1 GB) downloads on first use.

Step 3 — Environment files

- `ai/.env`: `MISTRAL_API_KEY` and `DATABASE_URL=postgresql://postgres:postgres@localhost:5433/digital_campus`
- `backend/.env`: same `DATABASE_URL`, a strong `SECRET_KEY`, `RAG_PIPELINE_HANDLER=ai.integration.answer_chat`, and (local only) `AUTO_VERIFY_EMAIL=true`.
- `frontend/.env.local`: `NEXT_PUBLIC_API_URL=http://localhost:8000`

Step 4 — Apply migrations, then ingest the knowledge base (once)

```
cd backend && alembic upgrade head && cd ..
python scripts/ingest_kb.py
```

Step 5 — Start backend and frontend

```
cd backend && PYTHONPATH=.. uvicorn app.main:app --port 8000 --reload
# in another terminal:
cd frontend && npm install && npm run dev
```

Open `http://localhost:3000` -> Register -> (auto-verified in dev) -> Login -> complete onboarding -> Chat.

4. Google Cloud setup for the Google Classroom integration

In plain words. To let students pull their Google Classroom courses and materials into the assistant, Google needs to know and trust our app. You register the app in Google Cloud, switch on the Classroom and Drive APIs, declare exactly what we want to read (read-only), and get a Client ID and secret the backend uses. Tokens we receive are stored encrypted.

Under the hood. The backend performs a standard OAuth 2.0 Authorization Code flow, requesting these read-only scopes:

- `classroom.courses.readonly`
- `classroom.coursework.me.readonly`
- `classroom.courseworkmaterials.readonly`
- `classroom.announcements.readonly`
- `drive.readonly`

The student is authenticated on the callback via a signed **state** JWT (10-minute TTL), and access/refresh tokens are encrypted at rest with a Fernet key.

Step-by-step

- 1 **Create / select a project:** Google Cloud Console -> project picker -> New Project (e.g. **digital-campus**).
- 2 **Enable APIs:** “APIs & Services” -> Enable APIs and Services -> enable **Google Classroom API** and **Google Drive API**.
- 3 **OAuth consent screen:** choose *External*; fill app name and support emails. Add the five read-only scopes above. While in *Testing*, add each tester's Google address under “Test users” (only they can authorise until the app is verified).
- 4 **Create credentials:** Credentials -> Create Credentials -> OAuth client ID -> Web application. Under *Authorized redirect URIs* add the two URIs shown below. Copy the Client ID and Client secret.

```
http://localhost:8000/courses/classroom/callback      (local dev)
https://YOUR_BACKEND_DOMAIN/courses/classroom/callback (production)
```

- 1 **Generate the token encryption key:**

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Then configure the backend (**backend/.env** locally, Secret Manager in production):

```
GOOGLE_CLASSROOM_CLIENT_ID=...apps.googleusercontent.com
GOOGLE_CLASSROOM_CLIENT_SECRET=...
GOOGLE_CLASSROOM_REDIRECT_URI=https://YOUR_BACKEND_DOMAIN/courses/classroom/callback
GOOGLE_TOKEN_ENCRYPTION_KEY=<the Fernet key>
```

Note. Leaving the client id/secret empty disables the integration — the manual “add a course + paste/upload materials” path keeps working. Going public beyond test users requires Google's OAuth verification because Classroom/Drive scopes are “sensitive/restricted”.

5. Deploying the platform on Google Cloud — step by step

In plain words. We host four things: the database (managed Postgres), a small cache (managed Redis), the backend, and the frontend. Managed database/cache mean Google runs them for us; the backend and frontend run as containers on Cloud Run, which scales automatically. Passwords and keys live in Google's Secret Manager, never in the code.

Target architecture

Component	Google service	Notes
Database	Cloud SQL for PostgreSQL 16	Enable the vector extension
Cache / rate limiting	Memorystore for Redis	Reached over the VPC connector
Backend (FastAPI)	Cloud Run	>=2 GB memory (torch + embedding model)
Frontend (Next.js)	Cloud Run	Or Firebase Hosting
Images	Artifact Registry	Built by Cloud Build
Secrets	Secret Manager	Injected as env vars into Cloud Run

Step 0 — Prerequisites

```
gcloud auth login
gcloud config set project digital-campus
gcloud services enable run.googleapis.com sqladmin.googleapis.com \
  redis.googleapis.com artifactregistry.googleapis.com cloudbuild.googleapis.com \
  secretmanager.googleapis.com vpcaccess.googleapis.com
gcloud artifacts repositories create app --repository-format=docker --location=europe-west1
```

Step 1 — Database (Cloud SQL + pgvector)

```
gcloud sql instances create fsb-db --database-version=POSTGRES_16 \
  --tier=db-custom-1-3840 --region=europe-west1
gcloud sql databases create digital_campus --instance=fsb-db
gcloud sql users set-password postgres --instance=fsb-db --password=STRONG_PASSWORD
# Connect once and enable the extension (or let 'alembic upgrade head' do it):
# gcloud sql connect fsb-db --user=postgres --database=digital_campus
# CREATE EXTENSION IF NOT EXISTS vector;
```

Step 2 — Redis (Memorystore) and a VPC connector

```
gcloud redis instances create fsb-redis --size=1 --region=europe-west1
gcloud compute networks vpc-access connectors create fsb-connector \
  --region=europe-west1 --range=10.8.0.0/28
```

Step 3 — Store secrets

```
printf 'STRONG_JWT_SECRET' | gcloud secrets create SECRET_KEY --data-file=-
printf 'YOUR_MISTRAL_KEY' | gcloud secrets create MISTRAL_API_KEY --data-file=-
printf 'FERNET_KEY' | gcloud secrets create GOOGLE_TOKEN_ENCRYPTION_KEY --data-file=-
printf 'OAUTH_CLIENT_SECRET' | gcloud secrets create GOOGLE_CLASSROOM_CLIENT_SECRET --data-file=-
```

Step 4 — Backend image and deploy

Add a **backend/Dockerfile** that installs both requirement sets and, ideally, pre-downloads the embedding model at build time so cold starts don't pay the ~1 GB fetch:

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y build-essential libpq-dev && rm -rf /var/lib/apt/lists/*
COPY backend/requirements.txt ai/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt -r ai/requirements.txt
COPY . .
RUN python -c "from sentence_transformers import SentenceTransformer; \
  SentenceTransformer('sentence-transformers/paraphrase-multilingual-mpnet-base-v2')"
ENV PYTHONPATH=/app
CMD ["sh", "-c", "cd backend && alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port $PORT"]

gcloud builds submit --tag europe-west1-docker.pkg.dev/digital-campus/app/backend

gcloud run deploy backend \
  --image europe-west1-docker.pkg.dev/digital-campus/app/backend \
  --region europe-west1 --memory 2Gi --cpu 2 --min-instances 1 \
  --vpc-connector fsb-connector \
  --add-cloudsql-instances digital-campus:europe-west1:fsb-db \
  --set-env-vars DATABASE_URL="postgresql://postgres:PW@digital_campus?host=cloudsql/digital-campus:europe-west1" \
  --set-env-vars REDIS_URL=redis://REDIS_IP:6379,FRONTEND_URL=https://YOUR_FRONTEND_URL \
  --set-env-vars RAG_PIPELINE_HANDLER=ai.integration:answer_chat \
  --set-env-vars GOOGLE_CLASSROOM_CLIENT_ID=...,GOOGLE_CLASSROOM_REDIRECT_URI=https://YOUR_BACKEND_URL/courses/classroom/callback \
  --set-secrets SECRET_KEY=SECRET_KEY:latest,MISTRAL_API_KEY=MISTRAL_API_KEY:latest,GOOGLE_TOKEN_ENCRYPTION_KEY=GOOGLE_TOKEN_ENCRYPTION_KEY:latest \
  --allow-unauthenticated
```

Do **not** set **AUTO_VERIFY_EMAIL** in production; configure real SMTP so email verification is the gate. **--min-instances 1** avoids cold-start model loads.

Step 5 — Frontend deploy

```
gcloud run deploy frontend \
  --source frontend --region europe-west1 \
  --set-env-vars NEXT_PUBLIC_API_URL=https://YOUR_BACKEND_URL \
  --allow-unauthenticated
```

Step 6 — Wire the URLs together

- Set the backend's **FRONTEND_URL** to the deployed frontend URL (CORS).
- Set the frontend's **NEXT_PUBLIC_API_URL** to the deployed backend URL.
- Add **https://YOUR_BACKEND_URL/courses/classroom/callback** to the OAuth client's Authorized redirect URIs (Section 4) and to **GOOGLE_CLASSROOM_REDIRECT_URI**.

6. Post-deployment checklist

- Logged-out visit to a protected URL redirects to `/login` (server-side).
- Register -> verify (real email) -> login -> onboarding -> dashboard shows real data.
- Chat streams a cited answer; a course-scoped chat works.
- Six rapid bad logins return HTTP 429 (Redis reachable).
- Cloud Run logs show timestamped application log lines.
- `alembic upgrade head` ran (schema + `vector` extension present).
- Google Classroom “Connect” completes and lists courses (tester account while unverified).

Deferred for a later hardening pass (documented in the app audit): httpOnly-cookie session migration, refresh tokens, password reset, and Google OAuth app verification for public use.